

DOCUMENTO TÉCNICO COMPLEMENTAR

Alimentador Automático IoT para Pets

Como o sistema funciona e como a solução foi construída

Projeto: PIBIC SISPROJ 59635 — Universidade do Estado do Amazonas

Curso: Engenharia de Controle e Automação

Orientação: Prof. Dr. Almir Kimura Júnior

Equipe: Caio Chaves, Eduardo Oliveira, Raysson Vieira, Moisés Bastos

Repositório: github.com/Caiodesiderio/alimentador-automatico-iot

Documento de apoio ao artigo “Developing an IoT Smart Pet Feeder: a case study on maker culture and AI integration in engineering education”.

1. O que o sistema faz

O sistema libera porções programadas de ração para um animal de porte médio, mede quanto foi efetivamente liberado e quanto o animal consumiu, e permite acompanhamento e acionamento remotos por aplicativo. Sobre o histórico acumulado, um pipeline em Python identifica a rotina alimentar e sinaliza desvios de consumo.

Quatro decisões distinguem esta implementação de um alimentador programável comum:

- **A porção é pesada, não contada.** A dosagem ocorre em malha fechada: o motor gira em blocos e a balança decide quando parar.
- **A célula de carga fica sob o comedouro,** e não sob o reservatório, o que torna o consumo do animal uma grandeza medida.
- **Os agendamentos rodam no dispositivo,** pelo relógio interno, sem depender de rede nem de o aplicativo estar aberto.
- **O desempenho da análise foi medido,** não presumido, sobre um conjunto de validação com anomalias rotuladas.



2. Arquitetura e fluxo de dados

O ponto que organiza toda a arquitetura é uma restrição de rede: a ESP32 fica atrás de tradução de endereços (NAT) na rede doméstica e **não aceita conexão vinda de fora**. O celular, portanto, nunca fala diretamente com o alimentador — nem estando na mesma Wi-Fi. Toda a comunicação é mediada pelo banco em nuvem, e quem inicia as conexões é sempre o dispositivo.

ELEMENTO	PAPEL	COMUNICAÇÃO
Aplicativo (celular do tutor)	Interface do tutor	Insere comandos no banco; lê o estado em tempo real
Supabase (PostgreSQL)	Fila de comandos e armazenamento	Recebe e serve dados; nunca inicia conexão com a ESP32

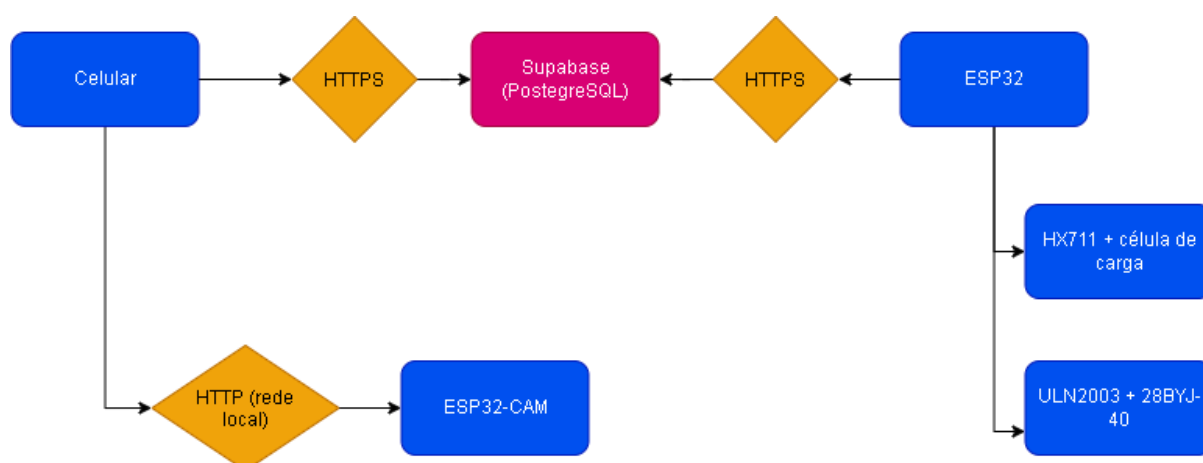
ELEMENTO	PAPEL	COMUNICAÇÃO
ESP32 (atrás de NAT)	Controle do dispositivo	Pergunta a fila a cada 3 s; envia peso e eventos a cada 30 s
HX711 + célula · ULN2003 + 28BYJ-48	Sensoriamento e atuação	Ligados diretamente à ESP32

Nenhuma seta entra na ESP32 vinda da internet — é ela quem inicia toda conexão.

Como “alimentar agora” funciona de ponta a ponta

1. O aplicativo insere uma linha na tabela commands com estado pendente.
2. A ESP32, que consulta a fila a cada 3 segundos, recebe o comando. A entrega é marcada na mesma transação da leitura — por isso o comando nunca é entregue duas vezes, e o animal não recebe porção dobrada se a resposta se perder.
3. O dispositivo executa a dosagem e registra o evento, com a massa realmente liberada.
4. O aplicativo, que acompanha o estado do comando, mostra o resultado.

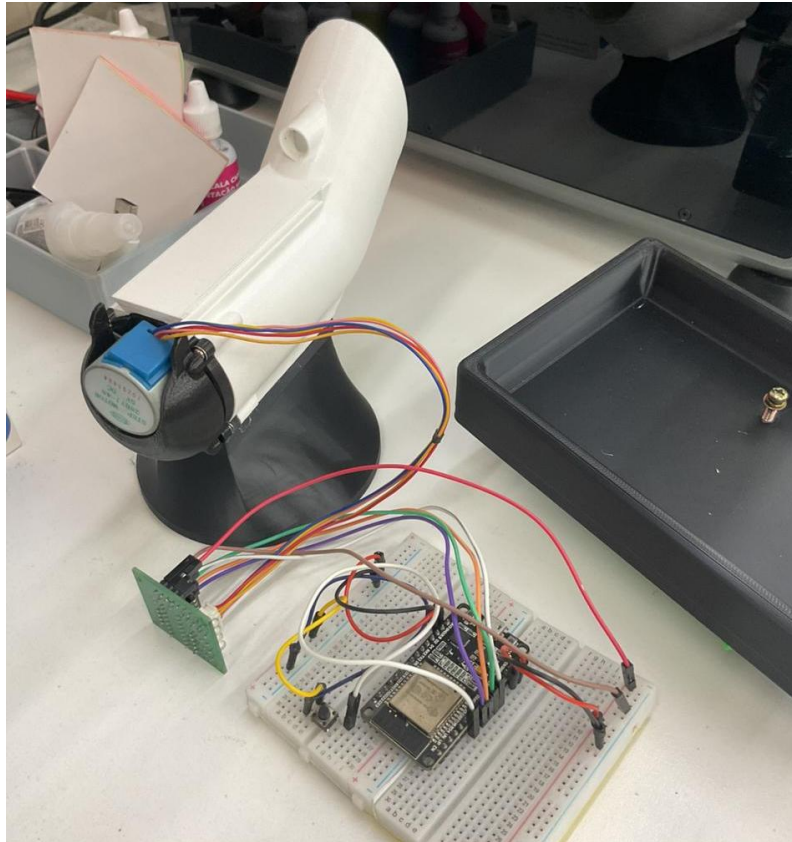
Comandos expiram em 2 minutos. Sem esse prazo, uma queda de rede de meia hora faria o alimentador despejar todas as ordens acumuladas assim que a conexão voltasse. Um comando não coletado a tempo é marcado como expirado e nunca executa.



3. Hardware

COMPONENTE	PINO	GPIO	OBSERVAÇÃO
HX711	DT / SCK	18 / 19	evita 16/17, usados pela PSRAM em módulos WROVER
HX711	VCC	3V3	nunca 5 V — a ESP32 não tolera 5 V na entrada
ULN2003	IN1–IN4	32, 33, 25, 26	fora dos pinos de flash e de strapping
ULN2003	VCC	5 V do step-down	picos de 300 mA derrubam o regulador da placa
LED de status	—	2	LED embutido da DevKit

A escolha dos GPIOs não é arbitrária: 6 a 11 são a memória flash, 34 a 39 são apenas de entrada, e 0, 12 e 15 são pinos de *strapping*, cujo nível no instante do boot altera o modo de inicialização. Uma bobina de motor ligada a um deles produz falha intermitente de boot — o tipo de defeito mais difícil de diagnosticar.



4. Dosagem em malha fechada

O fuso helicoidal impresso em 3D foi caracterizado por cálculo volumétrico antes de se escolher a estratégia de controle.

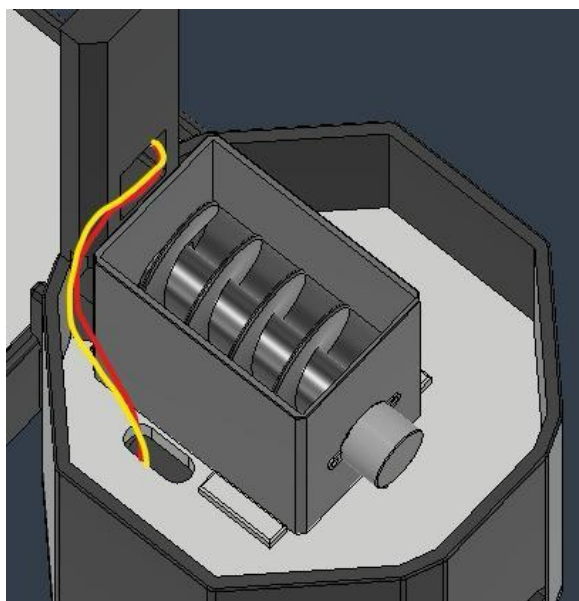
GRANDEZA	VALOR
Diâmetro externo da hélice	35 mm
Diâmetro do núcleo	14 mm
Passo da rosca	25 mm
Área útil da secção	8,08 cm ²
Volume teórico por volta	20,2 cm ³
Preenchimento real da calha	35 a 40 %
Densidade aparente da ração	0,38 g/cm ³
Massa por volta	≈ 3,0 g
Meio-passos por volta (28BYJ-48, redução 1:64)	4096

Por que malha aberta não funcionaria. A descarga não é contínua ao longo dos 360°: a maior parte cai no último quarto de volta. Somado ao travamento esporádico entre grãos, isso produz variação de $\pm 15\%$ a $\pm 20\%$ entre giros isolados — um giro solta 2,6 g, o seguinte 3,4 g. Em dez voltas, o erro acumulado passaria de 6 g, e o sistema não teria como saber.

Por isso o firmware gira em blocos curtos de aproximadamente 5 g, e entre um bloco e outro desenergiza as bobinas, aguarda 350 ms de acomodação e pesa a tigela. Na aproximação final os blocos caem para 1,5 g. O acionamento para quando a massa acumulada atinge a porção pedida dentro de 3 g de tolerância. Quatro blocos consecutivos sem ganho de peso são interpretados como obstrução, o que detecta rosca travada em cerca de oito segundos.

PORÇÃO	VOLTAS	TEMPO DE MOTOR	COM AS PAUSAS
20 g	6,7	36 s	~45 s
40 g	13,3	71 s	~85 s
120 g	40,0	213 s	~235 s

O tempo de dosagem é a principal limitação prática do protótipo, e ela é mecânica: não se resolve acelerando o motor, porque abaixo de 1000 μ s por meio-passo o 28BYJ-48 perde passos sob carga. As saídas são aumentar o passo ou o diâmetro do fuso, ou trocar o mecanismo.



5. Firmware

O firmware, escrito em C++ na Arduino IDE, mantém quatro rotinas periódicas e executa os agendamentos localmente.

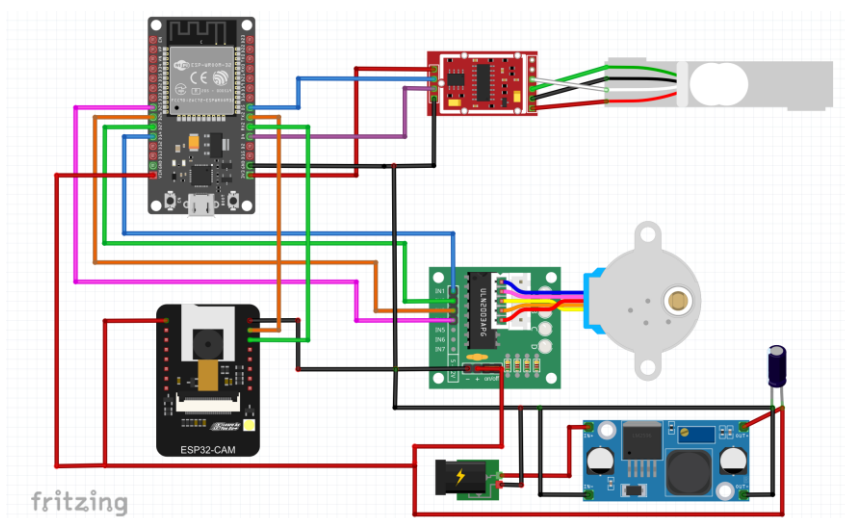
ROTINA	PERÍODO	FUNÇÃO
Fila de comandos	3 s	busca e executa ordens do aplicativo

ROTINA	PERÍODO	FUNÇÃO
Heartbeat	30 s	envia peso da tigela e sinaliza presença
Configuração	5 min	baixa agendamentos e calibração
Sincronização NTP	1 h	mantém o relógio interno acertado

Relógio sem RTC externo

O protótipo não tem relógio de tempo real com bateria, então o horário se perde a cada queda de energia. O firmware trata isso em três camadas: sincronização por NTP no boot e de hora em hora; a hora do servidor devolvida pelo próprio heartbeat como fonte reserva, para o caso de a rede bloquear a porta UDP 123, comum em rede institucional; e uma trava que impede a execução de agendamentos enquanto o relógio não estiver sincronizado. Pular uma refeição é preferível a liberar ração de madrugada.

Há ainda uma janela de tolerância de cinco minutos: se a energia voltar três minutos depois do horário marcado, a refeição acontece; quarenta minutos depois, não.



6. Banco de dados

PostgreSQL gerenciado pela plataforma Supabase, com esquema versionado em oito migrações SQL. As três últimas carregam o conjunto de dados e os resultados da análise, de modo que qualquer pessoa reproduz o mesmo estado do banco sem importação manual.

TABELA	CONTEÚDO
devices	alimentadores, com o segredo usado na verificação de credencial
pets	animal, peso e meta nutricional diária
schedules	agendamentos, baixados e executados pelo dispositivo
commands	fila de ordens do aplicativo para o dispositivo
food_readings	leituras periódicas do peso na tigela

TABELA	CONTEÚDO
feeding_events	uma linha por porção liberada, com o consumo medido
hopper_refills	reabastecimentos do reservatório, informados pelo tutor
analysis_runs · _clusters · _anomalies	saída do pipeline de análise

Segurança

O dispositivo não escreve diretamente em nenhuma tabela: toda escrita passa por funções que exigem um segredo próprio enviado em cabeçalho HTTP. As políticas de segurança em nível de linha restringem o acesso, e a chave administrativa da plataforma não aparece em nenhum arquivo do projeto. Está declarado como pendência que, por ainda não existir tela de autenticação, a leitura permanece liberada a quem detenha a chave pública — o caminho de correção está escrito no próprio arquivo de políticas.

Consumo medido, nível estimado

Como a célula de carga está sob o comedouro, o consumo é calculado por gatilho no banco a cada leitura de peso: é a queda do peso da tigela desde a liberação, limitada entre zero e a porção liberada. Já o nível do reservatório é estimado por balanço, e só volta ao valor correto quando o tutor registra o reabastecimento pelo aplicativo. A interface rotula esse número como estimado.

The screenshot shows the Supabase SQL Editor interface. The SQL query is as follows:

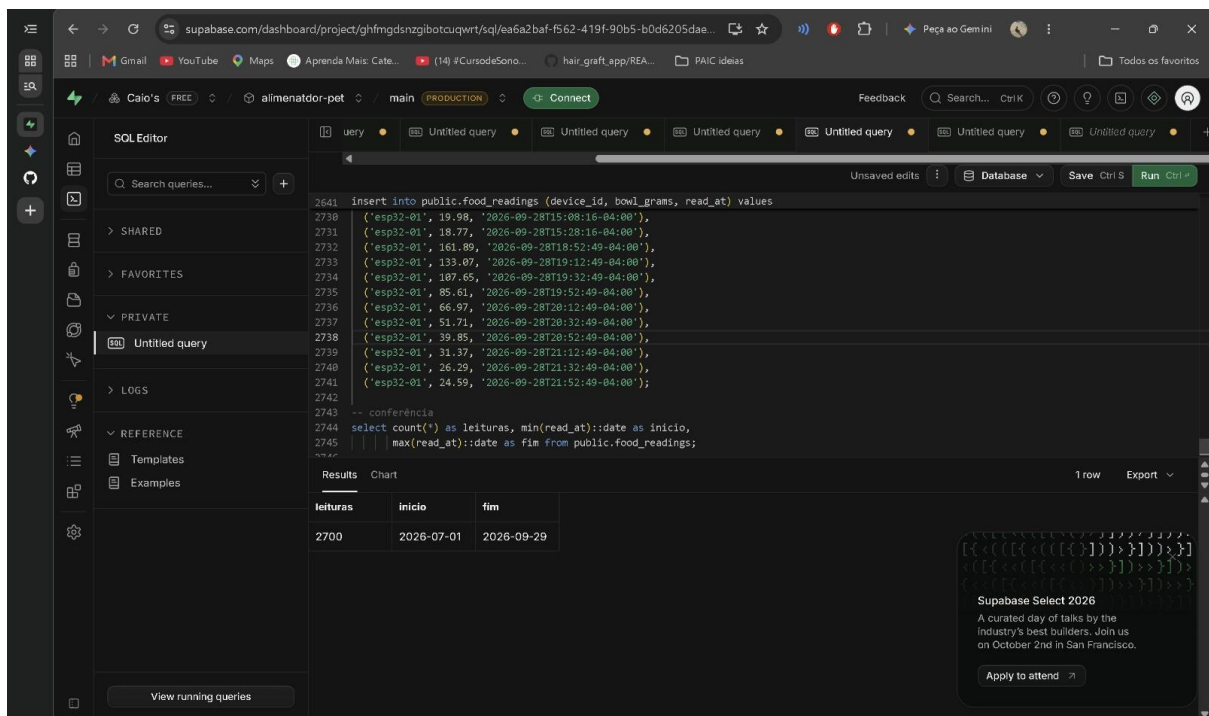
```

100 , grupos as (
101   select execucao_id, 'esp32-01', v.label, v.inicio::time, v.fim::time,
102         '1126,1143,1139,1144,1148,1143,1127,1139,1122,1150,1128,1138,1150,1133,1139,1158,1138,1148,1157,1135,1138,1158,1135,1129,1138,1158,1147,1128,
103         1139,1148,1147,1134,1162,1117,1128,1132,1133,1132,1122,1139,1140,1147,1132')
104   ) as v(label, inicio, fim, porcao, eventos, minutos)
105   returning 1
106 ),
107 desvios as (
108   insert into public.analysis_anomalies
109     (run_id, device_id, event_date, type, description, z_score, severity)
110   select execucao_id, 'esp32-01', v.data::date, v.tipo, v.descricao,
111         v.z, v.severidade
112   from execucao,
113        (values
114         ('2026-07-18', 'Consumo abaixo da média', 'Consumo 65% abaixo da média diária do período', -3.76, 'high'),
115         ('2026-08-01', 'Consumo abaixo da média', 'Consumo 56% abaixo da média diária do período', -3.259, 'high'),
116         ('2026-08-22', 'Consumo abaixo da média', 'Consumo 64% abaixo da média diária do período', -3.724, 'high'),
117         ('2026-09-18', 'Consumo abaixo da média', 'Consumo 66% abaixo da média diária do período', -3.817, 'high'),
118         ('2026-09-25', 'Consumo abaixo da média', 'Consumo 68% abaixo da média diária do período', -3.472, 'high')
119        ) as v(data, tipo, descricao, z, severidade)
120   returning 1
121 )

```

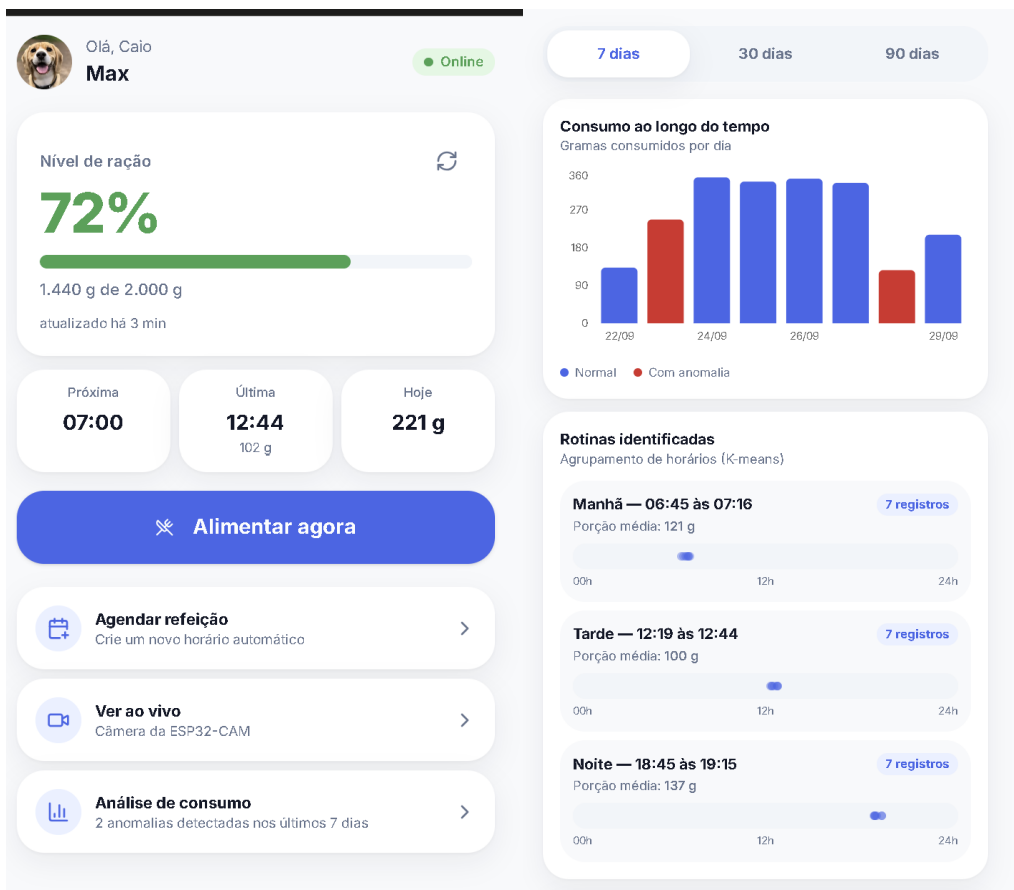
The results table shows the following data:

period_days	events_count	k_clusters	dataset_kind	grupos	anomalies
7	21	3	synthetic	3	1
30	93	4	synthetic	4	3
90	277	4	synthetic	4	5



7. Aplicativo

Aplicação web progressiva em React e TypeScript, instalável na tela inicial do celular, organizada em quatro telas: situação atual, agenda, câmera e análise. A comunicação com o banco usa assinatura de eventos em tempo real, de modo que uma leitura enviada pelo dispositivo aparece na tela sem recarregamento.



8. Conjunto de dados

Declaração. O conjunto utilizado é **integralmente sintético**. O protótipo está em bancada e nenhum animal foi monitorado. Os dados servem exclusivamente à validação técnica dos algoritmos e não sustentam qualquer conclusão sobre comportamento animal. A coluna `source` de cada registro distingue `synthetic` de `device`, e o aplicativo exibe o aviso correspondente enquanto houver registros sintéticos no período.

MÉTRICA	VALOR
Eventos de alimentação	277
Pesagens da tigela	2.700
Período coberto	2026-07-01 a 2026-09-28
Dias	90
Dias com anomalia injetada e rotulada	12
Consumo médio diário	331,28 g
Animais / dispositivos	1 / 1

A geração usa semente fixa, período em datas absolutas, identificadores determinísticos e apenas a biblioteca padrão do Python. Duas execuções produzem arquivos idênticos byte a byte. Os parâmetros da simulação — horários, dispersões, porções e consumo — derivam da caracterização mecânica da seção 4.

As doze anomalias foram inseridas em dias escolhidos, de três tipos: consumo reduzido, refeição não retirada e consumo elevado. **O rótulo de cada uma é salvo à parte**, e é isso que permite medir o acerto do detector em vez de avaliá-lo por inspeção do gráfico.

9. Análise de dados

Dois algoritmos clássicos, em Python com Scikit-learn. Não há rede neural, por escolha: o problema é de agrupamento e detecção de desvio em série pequena e interpretável, e um modelo de maior capacidade acrescentaria opacidade sem ganho.

K-means — rotina alimentar

Agrupa os horários das refeições, expressos em minutos desde a meia-noite. O número de grupos não é arbitrado: testam-se valores de 2 a 6 e adota-se o de maior coeficiente de silhueta. Sobre os 90 dias, o resultado foi **k = 4 com silhueta 0,946**: três refeições regulares e um quarto grupo, com sete ocorrências, correspondente a liberações eventuais.

GRUPO	QTD.	DESCRIÇÃO
Manhã	90	refeição regular, por volta de 07:00
Tarde	90	refeição regular, por volta de 12:30
Eventual	7	liberações eventuais, entre ~15:30 e ~16:45

GRUPO	QTD.	DESCRIÇÃO
Noite	90	refeição regular, por volta de 19:00

Agrupamento dos horários obtido por K-means (gráfico do documento original não reproduzido aqui; horários aproximados lidos do gráfico).

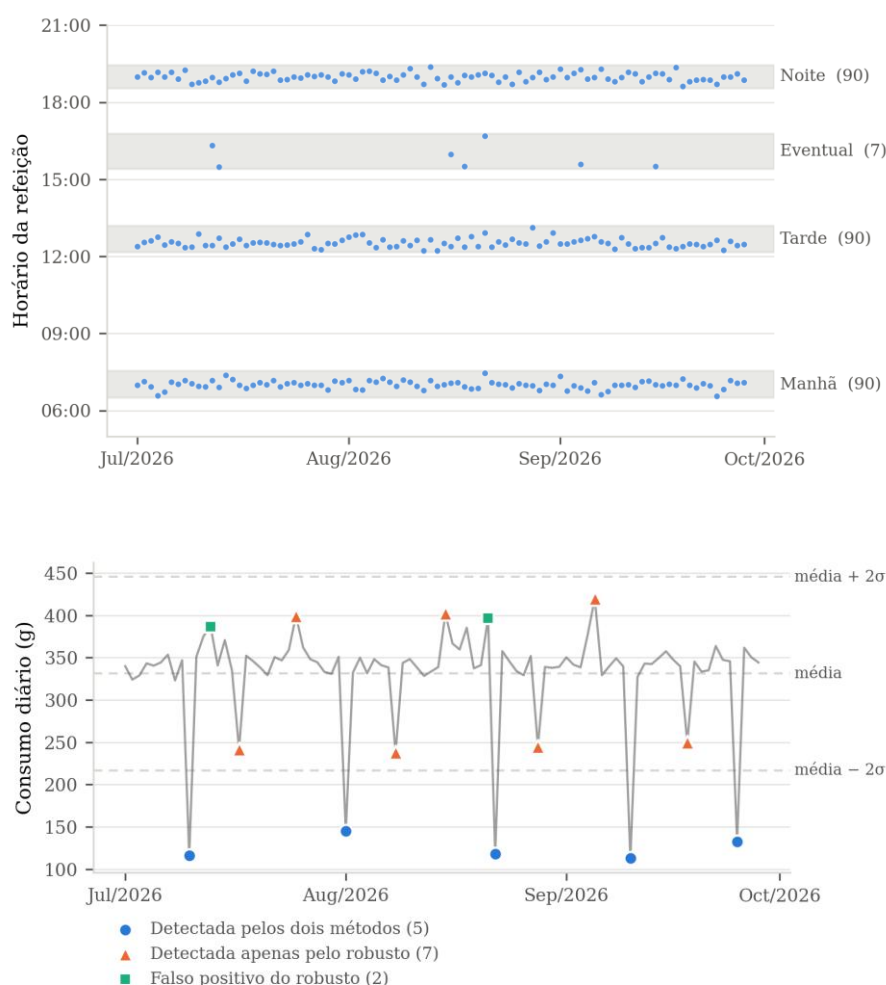
Z-score — anomalias de consumo

Sinaliza dias cujo consumo total se afasta da média mais de dois desvios-padrão. A avaliação contra os rótulos conhecidos produziu o achado central do trabalho:

DETECTOR	VP	FP	FN	PRECISÃO	REVOC.
Z-score clássico, $ z > 2,0$	5	0	7	1,00	0,42
Z-score robusto (mediana/MAD), $ z > 3,5$	12	2	0	0,86	1,00

F1: **0,59** para o Z-score clássico e **0,92** para o robusto.

Por tipo de anomalia, o clássico detectou 5 de 5 dias de consumo reduzido, mas **0 de 4** refeições não retiradas e **0 de 3** dias de consumo elevado. O robusto detectou todos.



Por que o clássico falha. A média e o desvio-padrão são calculados sobre a série que já contém as anomalias. Com 12 dias anômalos em 90, o desvio-padrão sobe para 57 g, bem acima da dispersão dos dias regulares, e o limiar de 2σ fica largo demais. As anomalias moderadas ficam escondidas dentro do desvio que elas mesmas inflaram — o efeito de mascaramento. Trocando média por mediana e desvio por MAD, a estatística de referência deixa de se mover com uma minoria de extremos, e as doze passam a ser detectadas.

A consequência prática foi adotar a variante robusta como detector principal, mantendo as duas calculadas para comparação. A limitação foi identificada **antes do início da coleta real**, quando corrigir ainda é barato — o que justifica, por si só, a construção de um conjunto de validação com anomalias rotuladas.

10. Reprodutibilidade e verificação independente

Nenhuma afirmação deste documento depende de acesso à instância de banco do projeto. Tudo é reproduzível a partir do repositório público:

- **Automática.** A cada envio de código, o GitHub executa a verificação: confere os SHA-256 dos arquivos publicados, regeira o conjunto do zero e compara byte a byte, confirma que os números da documentação saem do pipeline, e aplica as oito migrações em um PostgreSQL limpo conferindo as contagens resultantes. O log é público.
- **Local.** `python analise/verificar.py` executa as mesmas conferências.
- **Banco do zero.** Um contêiner PostgreSQL e as oito migrações reproduzem o estado completo em cerca de dez minutos.

11. Limitações declaradas

1. Os dados de análise são sintéticos; nenhuma conclusão sobre comportamento animal real decorre deles.
2. Um único animal e um único dispositivo — sem variabilidade entre indivíduos ou ambientes.
3. Anomalias injetadas de apenas três tipos; anomalias reais tendem a ser mais sutis, de modo que o desempenho medido é provavelmente otimista.
4. Ruído gaussiano e independente, sem deriva térmica da célula nem acomodação mecânica da estrutura impressa.
5. O tempo de dosagem decorrente da vazão do fuso restringe o uso prático na configuração atual.
6. A camada de autenticação de usuários ainda não foi implementada.
7. O módulo de câmera não foi integrado.

12. Onde está cada coisa no repositório

CAMINHO	CONTEÚDO
DATASET.md	documentação completa do conjunto de dados
supabase/migrations/	esquema, políticas, funções e carga do banco
firmware/alimentador_esp32/	firmware da ESP32, comentado em português
app/	aplicativo completo
analise/gerar_dataset.py	geração do conjunto sintético
analise/pipeline_analise.py	K-means e Z-score
analise/verificar.py	verificação independente
docs/dosagem-mecanica.md	memória de cálculo da seção 4
docs/VERIFICACAO.md	como conferir tudo sem falar com os autores